

Intégration des SPC-150N dans le GUI

Sep 30, 2026 · @Elias

Le logiciel DCC garde les détecteurs ; ton GUI pilote les SPC-150N par FLIMCore, un module que je construis sans voir ton code.

Démarrage et arrêt

Le logiciel DCC et le GUI tournent en même temps ; SPCM doit rester fermé, car il verrouille les SPC-150N.

1. Allumer les châssis Magma et Simple-Tau, puis le PC.
2. Ouvrir le logiciel DCC autonome, pas le panneau DCC de SPCM. Régler M1 (C1 et C3 à 82 %, b0, refroidissement 5 V et 1,98 A), puis cliquer « Enable outputs ».
3. Lancer le GUI. FLIMCore initialise les SPC-150N et vérifie :
 - modules 0 et 1 prêts, n° de série 3N0317 et 3N0318 ;
 - SYNC correct sur les deux cartes ;
 - réglages appliqués conformes à `reglages_spc.jl` (le tableau « demandé → appliqué » des scripts) ;
 - taux CFD au-dessus d'un seuil, laser allumé. Sinon : « Détecteurs éteints ? Enable outputs dans le logiciel DCC ».
4. Pendant les mesures, le GUI surveille le taux CFD : une chute brutale signale une coupure par surcharge dans le logiciel DCC.
5. Fermer le GUI : FLIMCore arrête la mesure et libère les cartes, même après une erreur. Couper ensuite les sorties dans le logiciel DCC.

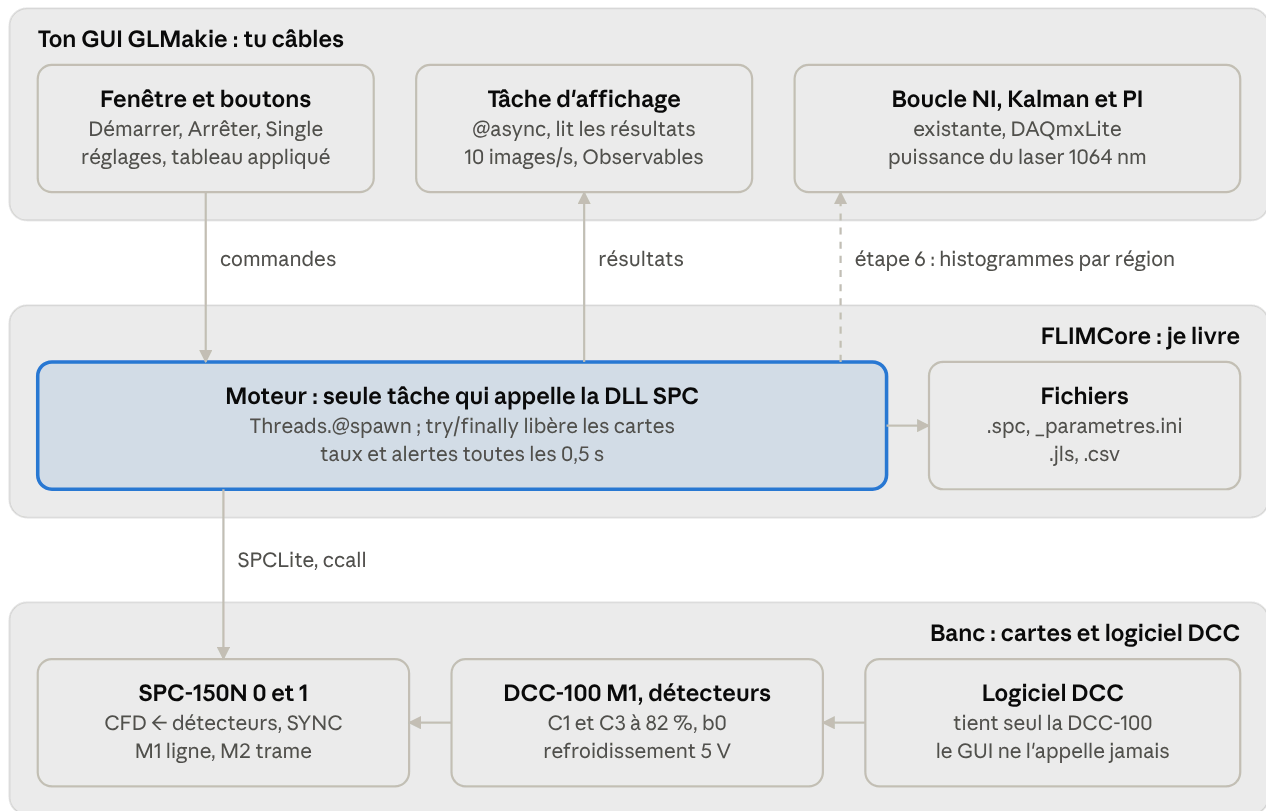
Le GUI n'appelle jamais la DLL des DCC-100 : DCC_init échoue (état -4) tant que le logiciel DCC tient les modules, et couperait les sorties dès qu'il ne les tient plus. DCCLite ne sert plus qu'à l'inventaire et au déverrouillage, logiciel DCC fermé.

Architecture

FLIMCore se glisse entre ton GUI et les cartes ; le logiciel DCC reste à part, et rien dans le GUI ne l'appelle.

Seul le moteur touche les SPC-150N ; le logiciel DCC garde les détecteurs

Le GUI envoie des commandes et lit des résultats ; il n'appelle aucune DLL



architecture · 3 couches, 2 canaux

Les flèches pleines existent dès l'étape 3 ; la flèche pointillée arrive avec la lecture du clamp, à l'étape 6.

- **Moteur** : un fil à part (`Threads.@spawn`), seul à appeler la DLL SPC ; il reçoit les commandes et publie les résultats par deux `Channel` .
- **Tâche d'affichage** : sur le fil du GUI, elle lit les résultats, garde la dernière trame et met à jour les Observables au plus 10 fois par seconde.
- **Taux** : le moteur les relit toutes les 0,5 s ; c'est le seul signe, côté GUI, que le logiciel DCC a bien allumé les détecteurs.
- **Sources de photons** : cartes, rejeu d'un .spc, plus tard la SPC-QC-104, derrière la même interface ; le GUI ne voit pas la différence.

Interface entre le GUI et FLIMCore

Le GUI ne fait aucun ccall : il envoie des commandes au moteur et lit ses résultats, par deux canaux (Channel).

Appel côté GUI	Ce qu'il fait	Ce qui revient
<code>m = demarrer_moteur(reglages)</code>	lance la tâche qui possède les cartes, puis SPC_init	la poignée <code>m</code>
<code>verifier(m)</code>	modules, n° de série, SYNC, CFD, tableau demandé → appliqué	un <code>EtatCartes</code>
<code>commander!(m, Imagerie(geometrie; duree = Inf))</code>	FIFO avec les horloges du scanner, en continu ou pour une durée	une <code>ImageTrame</code> par trame et par carte
<code>commander!(m, Single(temps_s, n))</code>	histogrammes construits dans la carte	un <code>HistoSingle</code> par mesure
<code>commander!(m, Arret())</code>	arrête la mesure en cours	un <code>Fin</code>
<code>take!(m.resultats)</code>	lit tout ce que produit le moteur	<code>ImageTrame</code> , <code>HistoSingle</code> , <code>Taux</code> , <code>Alerte</code> , <code>Fin</code>
<code>arreter_moteur(m)</code>	arrête, libère les cartes, termine la tâche	rien

Contenu des résultats :

- `ImageTrame` : carte, n° de trame, intensité, somme des temps d'arrivée, déclin, photons, pertes, débordement du FIFO ;
- `Taux` : SYNC, CFD, TAC, ADC et remplissage du FIFO par carte, toutes les 0,5 s, mesure en cours ou non ;
- `Alerte` : gravité et texte (SYNC perdu, CFD à zéro, FIFO débordé, peigne de l'ADC, carte verrouillée, réglage non appliqué).

Étapes

Chaque étape se termine par un test qui passe avant la suivante ; je ne vois jamais le code du GUI, je livre FLIMCore et un gabarit d'adaptateur, tu câbles.

Étape	Contenu	Qui	Test de passage
0. Banc validé	Scripts actuels avec les réglages SPCM : tableau demandé → appliqué propre, pas de peigne, image alignée	toi, je corrige	Image et Single conformes à SPCM
1. FLIMCore	Module tiré des scripts : fonctions pures, aucun affichage ni exécution au chargement ; les scripts deviennent de simples lanceurs	moi	Mêmes sorties que les scripts sur un .spc rejoué
2. Traitement trame par trame	Rangement incrémental des photons, source « rejou » qui relit un .spc à vitesse réelle	moi	Rejou : résultat identique au traitement en bloc, sans les cartes
3. Moteur	Tâche seule propriétaire de la DLL, canaux commandes et résultats, taux et alertes	moi	100 démarrages et arrêts, une erreur simulée, un Ctrl+C : cartes toujours libérées
4. Adaptateur GLMakie	Observables, boutons Démarrer et Arrêter, affichage limité à 10 Hz, fermeture propre	toi, sur mon gabarit	10 min d'imagerie continue sans gel ni perte ; après fermeture, aucune carte verrouillée
5. Panneau Single et enregistrement	Histogrammes Single, sauvegarde des données brutes et des paramètres relus	toi et moi	CSV identique au script

Étape	Contenu	Qui	Test de passage
6. Lecture du clamp	FIFO avec routage (tests 4 et 5), un histogramme par région et par passe, remis à ton Kalman et à ton PI	moi et toi	Latence mesurée inférieure à une passe
7. SPC-QC-104	Nouvelle source de photons derrière la même interface	moi	GUI inchangé

Règles de sécurité et de robustesse

Ces règles viennent des pannes déjà vues sur le banc : violations d'accès, verrous restés posés, modules périmés dans Julia.

- **Une seule tâche touche la DLL SPC.** Un bouton envoie une commande ; aucun ccall dans un callback GLMakie.
- **Modules détectés et prêts seulement.** Un appel sur un module absent ou non initialisé peut tuer Julia, try/catch n'y peut rien.
- **Libération garantie.** try/finally dans le moteur, fermeture de la fenêtre (`events(fig).window_open`) et `atexit` appellent tous `arreter_moteur` .
- **Jamais de reprise forcée en silence.** Un bouton « Déverrouiller » demande confirmation, SPCM fermé.
- **Observables mis à jour par la tâche du GUI seulement.** Une tâche `@async` lit le canal et ne garde que la dernière trame : 31,25 trames/s arrivent, 10 images/s s'affichent.
- **Julia lancé avec plusieurs fils** (`julia -t auto`), FLIMCore chargé au niveau supérieur avant la fenêtre, jamais par `include` dans un callback.
- **Mises à jour du code :** Revise pendant le développement, sinon redémarrer Julia ; la garde de version refuse un module périmé.
- **Chaque acquisition garde sa trace :** flux brut, paramètres relus dans la carte, géométrie de l'image, et réglages DCC déclarés, que le GUI ne peut pas lire.
- **Un seul fichier de réglages,** relu à chaque démarrage de mesure ; le GUI peut l'éditer et le réécrire.

Questions ouvertes

Tes réponses fixent l'étape 3 ; aucune ne demande de me montrer du code.

- ☐ Ton GUI tourne-t-il déjà avec plusieurs fils (`julia -t`), et la boucle NI vit-elle dans sa propre tâche ?
- ☐ Le clamp utilisera-t-il ces mêmes SPC-150N, par routage, et l'imagerie alternera-t-elle avec lui ou tournera-t-elle en parallèle ?
- ☐ Quels affichages en direct : intensité, temps moyen, déclin, taux ?
- ☐ Où vivent tes réglages aujourd'hui : un fichier, un panneau du GUI ?
- ☐ Quel format de sauvegarde pour tes analyses : .spc et .jls, ou .sdt pour ton lecteur existant ?